

Assignment2-C Angular Notes

1. What this project is

This is a small Angular app with three pages:

- Login
- Register
- Dashboard

It uses:

- Angular routing to switch between pages
- Reactive forms for input handling and validation
- localStorage to save user data in the browser
- An auth guard to protect the dashboard page

2. How the app starts

The app starts from src/main.ts.

main.ts bootstraps AppModule, which means Angular loads the root module of the project.

Important idea:

- main.ts is the entry point
- AppModule is the main container of the app

3. AppModule

File: src/app/app.module.ts

AppModule tells Angular:

- which components belong to the app
- which Angular modules should be available
- which component should load first

Declarations:

- AppComponent
- LoginComponent
- RegisterComponent
- DashboardComponent

Imports:

- BrowserModule
- AppRoutingModule
- FormsModule
- ReactiveFormsModule

Bootstrap:

- AppComponent

Important idea:

- declarations = your components
- imports = Angular features you want to use
- bootstrap = starting component

4. AppComponent

File: src/app/app.component.html

This file contains:

- `<router-outlet></router-outlet>`

`router-outlet` is the place where Angular shows the current page based on the route.

Important idea:

- `AppComponent` is the shell
- `router-outlet` is the display area for routed components

5. Routing

File: `src/app/app-routing.module.ts`

Routes used in this app:

- `/login` -> `LoginComponent`
- `/register` -> `RegisterComponent`
- `/home` -> `DashboardComponent`
- `'` redirects to `/login`

The `/home` route uses `AuthGuard`.

Important idea:

- routing decides which component appears
- `redirectTo` sends the user to a default page
- `canActivate` protects a route

6. AuthGuard

File: `src/app/auth.guard.ts`

`AuthGuard` checks whether the user is logged in before opening the dashboard.

Logic:

- Read `isLoggedIn` from `localStorage`
- If it is not `"true"`, navigate to login
- Return `false`
- Otherwise return `true`

Important idea:

- guards are used to block unauthorized access to pages

7. Register page

File: `src/app/components/register/register.component.ts`

This component creates a reactive form with:

- `name`
- `email`

Validators:

- `name` is required
- `email` is required
- `email` must be valid

When `registerSubmit()` runs:

1. Read existing users from `localStorage` using the key `registeredUsers`
2. Convert the JSON string into an array
3. Check whether the email already exists

4. If yes, show an error
5. If no, add the new user object
6. Save the updated array back to localStorage
7. Move the user to the login page

Important idea:

- localStorage stores only strings
- arrays and objects must be converted using `JSON.stringify()`
- stored JSON must be read back using `JSON.parse()`

8. Login page

File: `src/app/components/login/login.component.ts`

This component also creates a reactive form with:

- name
- email

When `loginSubmit()` runs:

1. Read `registeredUsers` from `localStorage`
2. Convert it into an array
3. Search for a matching user by name and email
4. If found:
 - set `isLoggedIn` to `true`
 - save name
 - save email
 - navigate to `/home`
5. If not found:
 - show `loginError`

Important idea:

- login is not checking a database
- it only compares values stored in the browser

9. Dashboard page

File: `src/app/components/dashboard/dashboard.component.ts`

When the dashboard loads:

- `userName` is read from `localStorage`
- `userEmail` is read from `localStorage`
- `nameInitial` is created from the first letter of the name
- `joinDate` is created using the current date

Logout:

- remove `isLoggedIn`
- remove name
- remove email
- navigate back to `/login`

Important idea:

- dashboard data is coming from browser storage
- logout clears the stored session values

10. Template syntax used in Angular

Examples from the project:

- `[formGroup]="loginForm"`

Binds the HTML form to the FormGroup created in TypeScript

- `formControlName="email"`
Connects an input element to a specific form field
- `(click)="loginSubmit()"`
Runs a TypeScript method when the button is clicked
- `*ngIf="loginError"`
Shows an element only when the condition is true
- `{{ userName }}`
Displays a TypeScript variable in HTML

Important idea:

- `[]` is property binding
- `()` is event binding
- `{{ }}` is interpolation
- `*ngIf` is structural rendering

11. Full user flow

1. App starts from main.ts
2. AppModule loads
3. AppComponent displays router-outlet
4. Empty route redirects to /login
5. User can go to register page
6. Register page saves user data in localStorage
7. User logs in with name and email
8. Login stores session values in localStorage
9. User enters /home
10. AuthGuard allows access because isLoggedIn is true
11. Dashboard shows stored user details
12. Logout clears session data and returns to login

12. How to explain this in an exam

You can say:

"This project is an Angular authentication demo using routing, reactive forms, localStorage, and route guards. The register page stores users in localStorage, the login page validates the entered credentials against stored users, the auth guard protects the dashboard route, and the dashboard displays user data from localStorage."

13. Key terms to remember

- bootstrap: starts the Angular app
- module: groups related Angular features
- component: controls one part of the UI
- route: maps a URL path to a component
- guard: controls whether a route can open
- reactive form: form created and validated in TypeScript
- localStorage: browser storage for key-value string data

14. Quick summary

This project does not use a backend or database.

It simulates authentication fully in the browser.

That makes it simple for learning Angular concepts such as:

- components
- routing
- guards
- reactive forms
- localStorage